

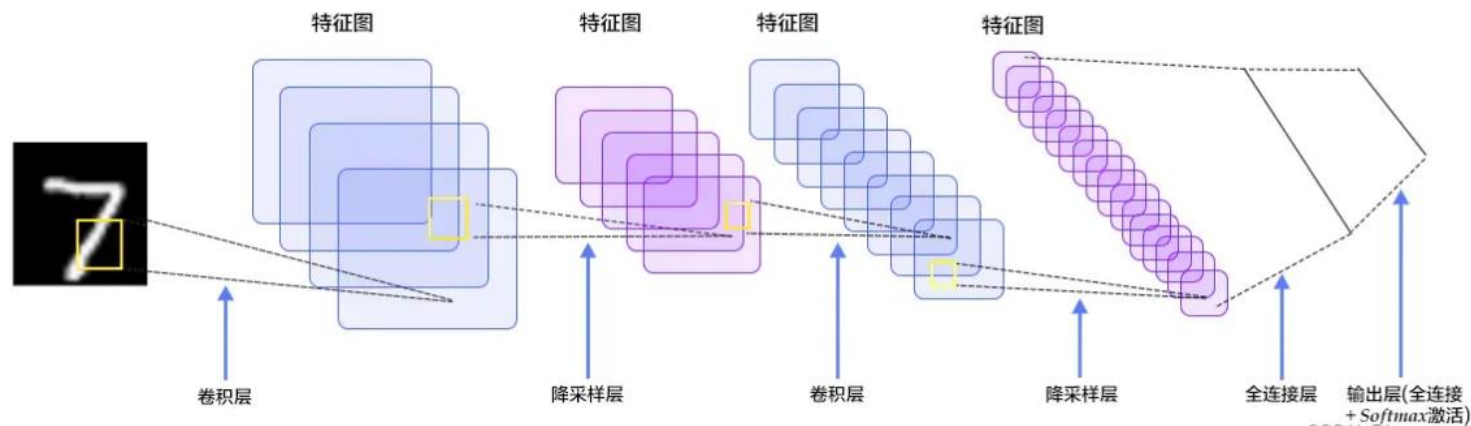
Computer Vision: Machine Learning based Facial Recognition, Autonomous Driving, and Image Processing

TA: Shujian Yu

Date: 2026/04/05

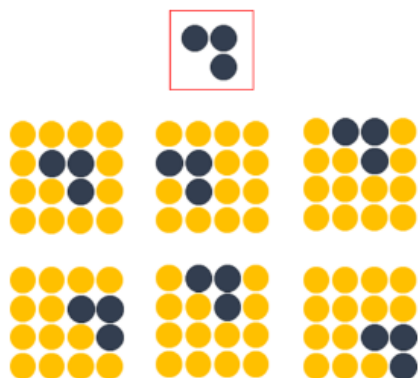
什么是卷积神经网络 (CNN)

卷积神经网络 (Convolutional Neural Networks, CNN) 这个概念的提出可以追溯到二十世纪 80~90 年代，但是有那么一段时间这个概念被“雪藏”了，因为当时的硬件和软件技术比较落后，而随着各种深度学习理论相继被提出以及数值计算设备的高速发展，卷积神经网络得到了快速发展。那究竟什么是卷积神经网络呢？以手写数字识别为例，整个识别的过程如下所示：



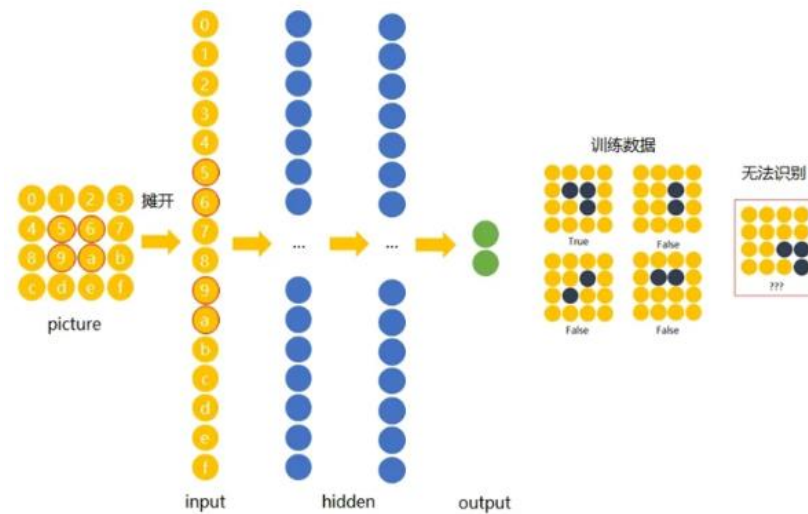
- 输入层：输入图像等信息
- 卷积层：用来提取图像的底层特征
- 池化层：防止过拟合，将数据维度减小
- 全连接层：汇总卷积层和池化层得到的图像的底层特征和信息
- 输出层：根据全连接层的信息得到概率最大的结果

为什么要卷积神经网络 (CNN)



传统神经网络

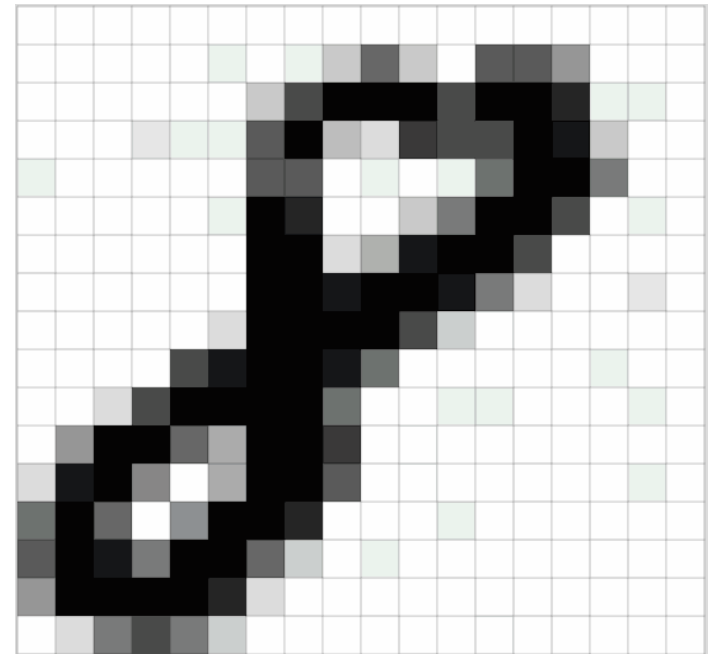
CNN

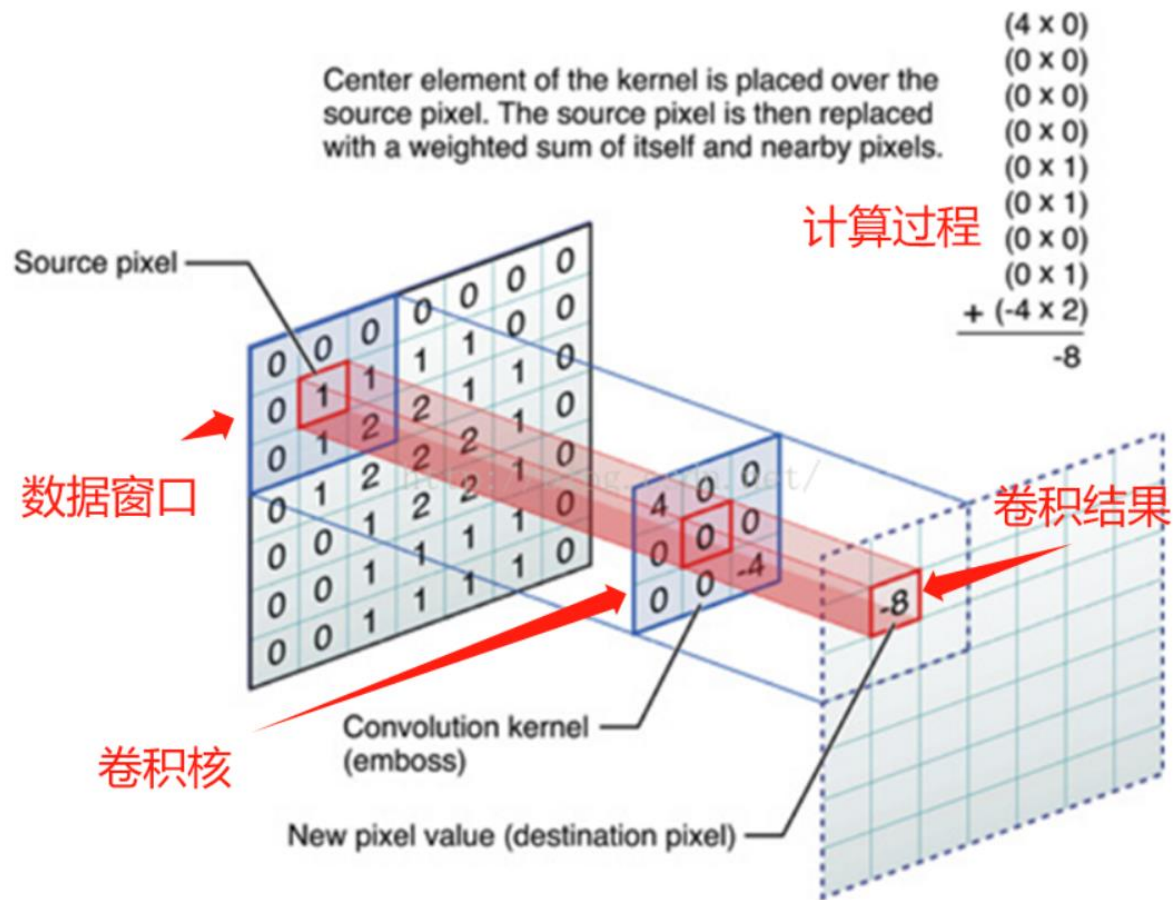


输入层比较简单，这一层的主要工作就是输入图像等信息，因为卷积神经网络主要处理的是图像相关的内容，但是我们人眼看到的图像和计算机处理的图像是一样的么？

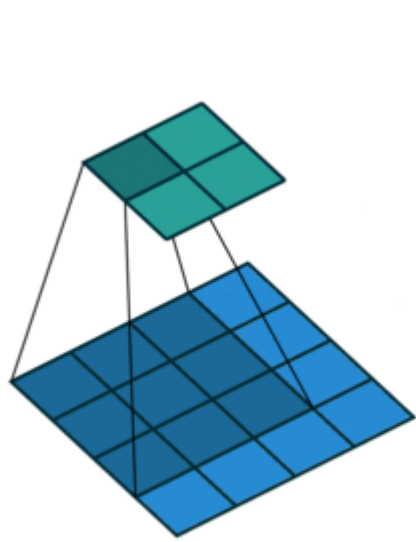
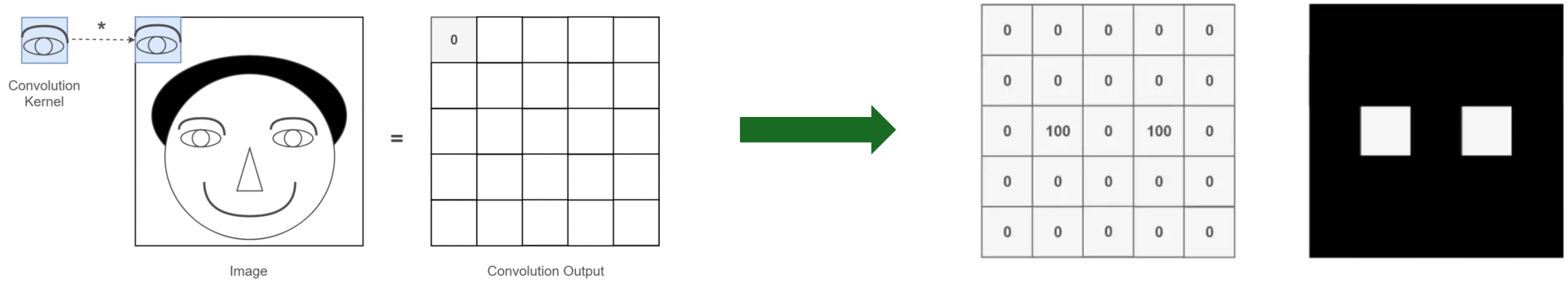
右图又称为灰度图像，因为其每一个像素值的范围是0~255（由纯黑色到纯白色），表示其颜色强弱程度。另外还有黑白图像，每个像素值要么是0（表示纯黑色），要么是255（表示纯白色）。

我们日常生活中最常见的是RGB图像，有三个通道，分别是红色、绿色、蓝色。每个通道的每个像素值的范围也是0~255，表示其每个像素的颜色强弱。但我们日常处理的基本都是灰度图像，因为比较好操作（值范围较小，颜色较单一），有些RGB图像在输入给神经网络之前也被转化为灰度图像，也是为了方便计算，否则三个通道的像素一起处理计算量非常大。

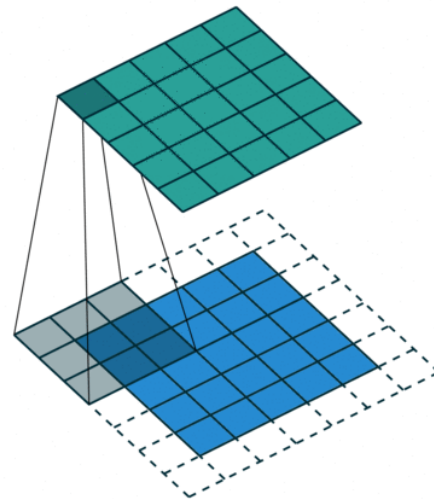




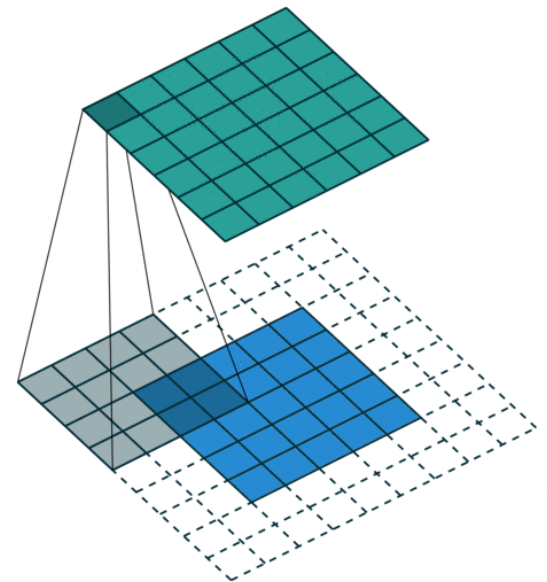
在卷积神经网络中，卷积操作是指将一个可移动的小窗口（称为数据窗口）与图像进行逐元素相乘然后相加的操作。这个小窗口其实是一组固定的权重，它可以被看作是一个特定的滤波器（filter）或卷积核。这个操作的名称“卷积”，源自于这种元素级相乘和求和的过程。这一操作是卷积神经网络名字的来源。



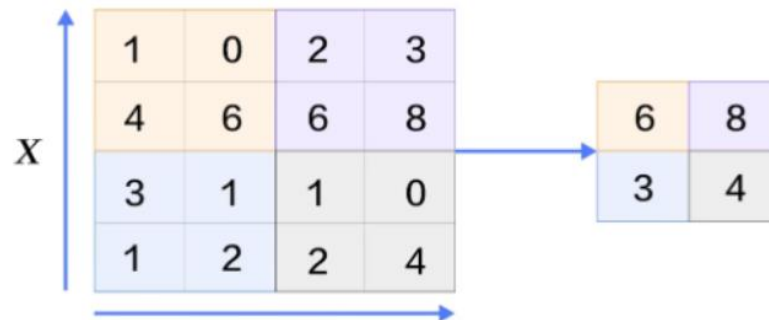
正常卷积



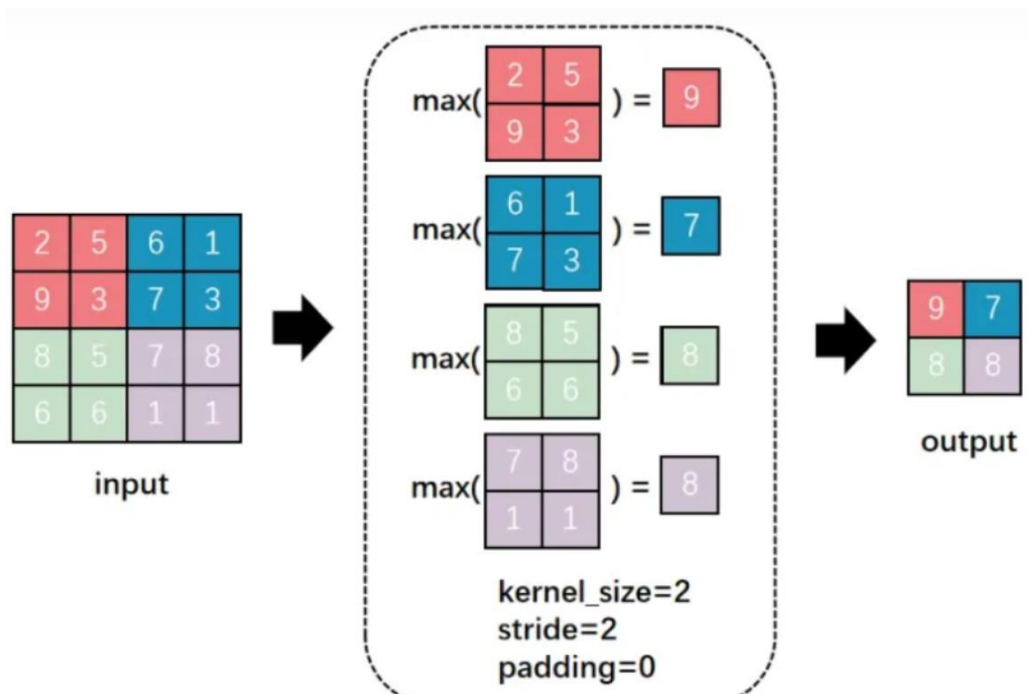
Padding为1时卷积的过程



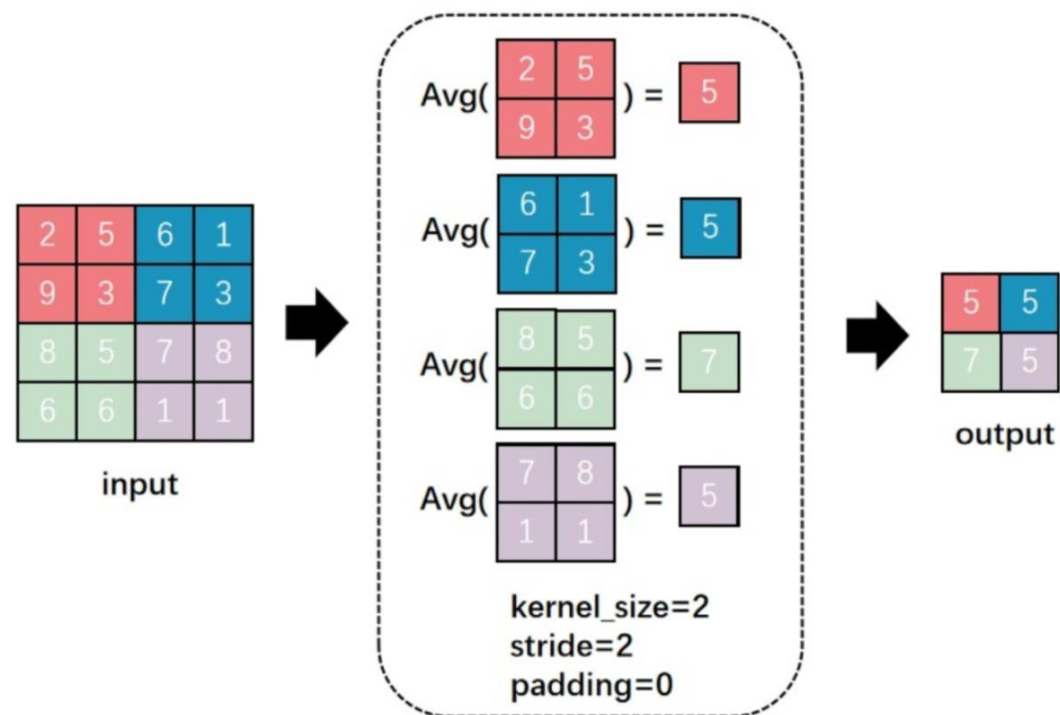
Padding为2时卷积的过程



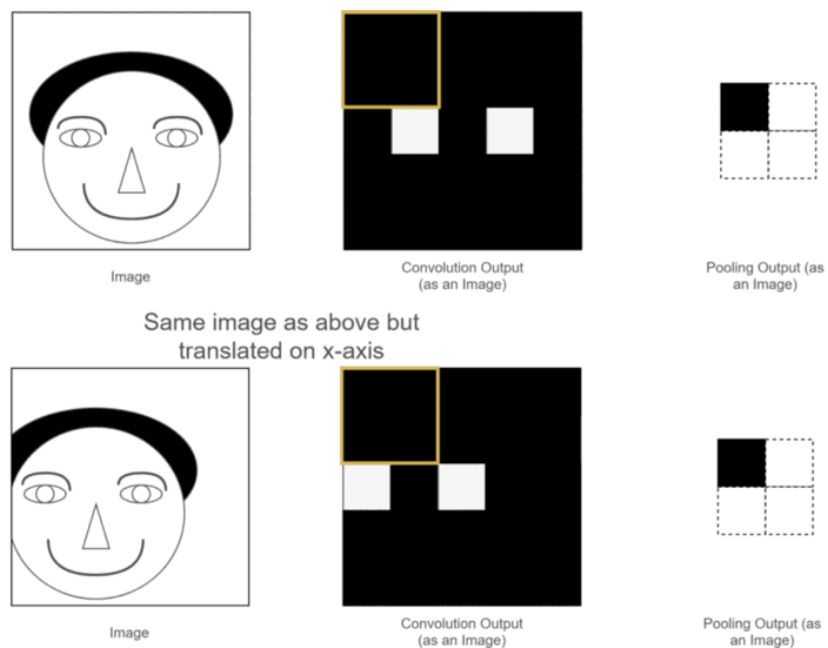
池化



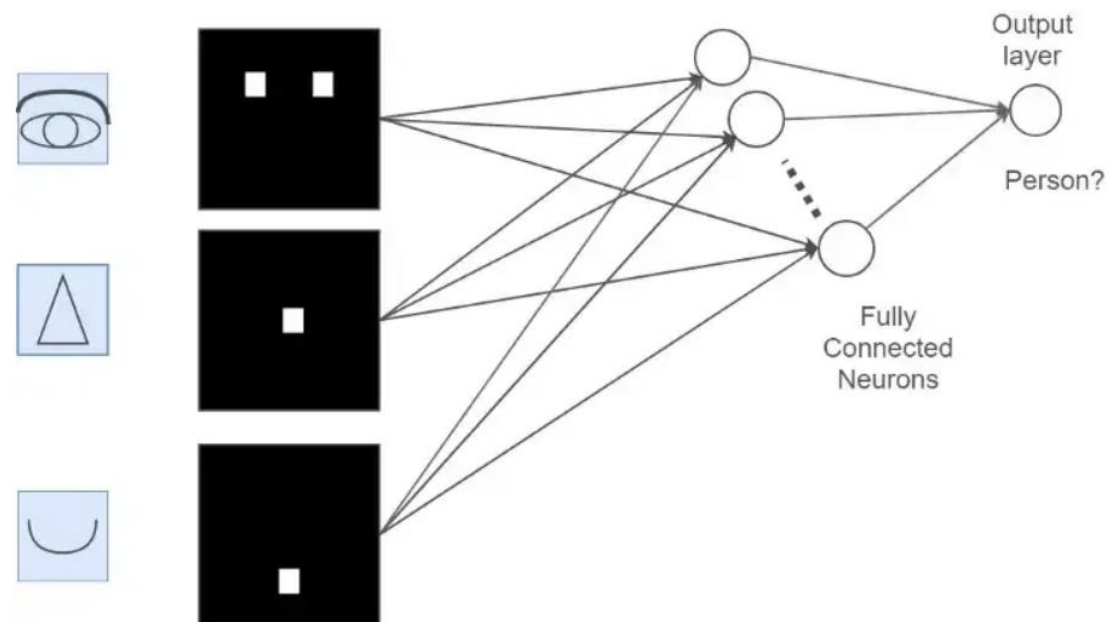
最大池化



平均池化

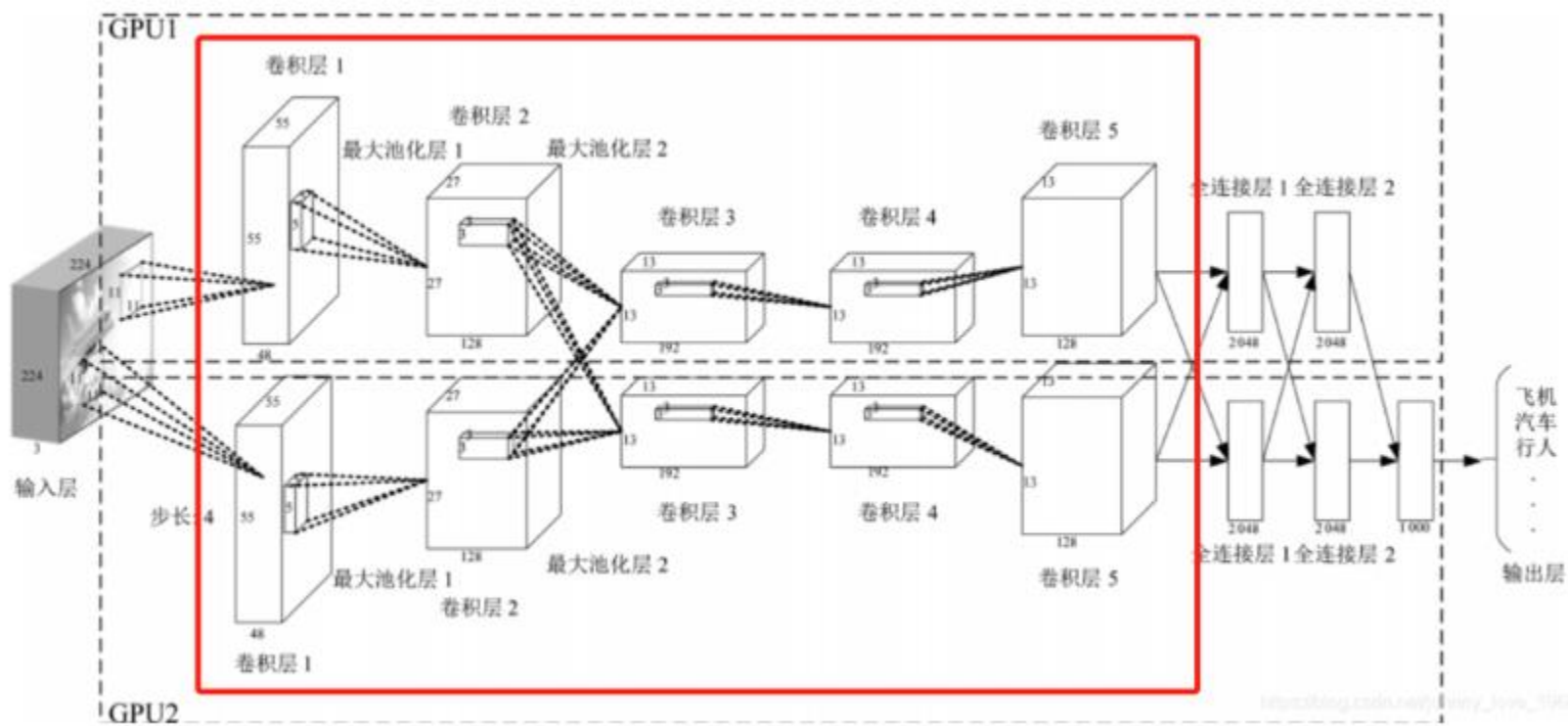


池化的平移不变性



全连接的过程

整体过程



☐ FCN?

☐ RNN?

☐ 123456789N?

☐N?

实验一：MNIST二值分割（二分类（像素级））--FCN简化版

```
class MiniUNet(nn.Module):
    def __init__(self):
        super().__init__()
        # 编码器（无下采样）
        self.encoder = nn.Sequential(
            nn.Conv2d(1, 8, 3, padding=1), # 输入通道1, 输出通道8
            nn.ReLU(),
            nn.Conv2d(8, 16, 3, padding=1), # 输出通道16
            nn.ReLU()
        )
        # 解码器
        self.decoder = nn.Sequential(
            nn.Conv2d(16, 8, 3, padding=1), # 输入通道16, 输出通道8
            nn.ReLU(),
            nn.Conv2d(8, 1, 3, padding=1), # 输出通道1
            nn.Sigmoid() # 输出概率图
        )

    def forward(self, x):
        x = self.encoder(x) # [B, 1, 28, 28] → [B, 16, 28, 28]
        x = self.decoder(x) # [B, 16, 28, 28] → [B, 1, 28, 28]
        return x
```

实验二：MNIST边缘检测回归（像素级）--类似经典的分类CNN

```
class EdgeDetector(nn.Module):
    def __init__(self):
        super().__init__()
        self.layers = nn.Sequential(
            nn.Conv2d(1, 16, 3, padding=1), # 输入通道1, 输出通道16
            nn.ReLU(),
            nn.Conv2d(16, 32, 3, padding=1), # 输出通道32
            nn.ReLU(),
            nn.Conv2d(32, 1, 3, padding=1), # 输出通道1
            nn.Sigmoid()
        )

    def forward(self, x):
        return self.layers(x) # [B, 1, 28, 28] → [B, 1, 28, 28]
```

Computer Vision: Machine Learning based Facial Recognition, Autonomous Driving, and Image Processing

TA: Shujian Yu

Date: 2026/04/05